UNIVERSITY OF LEEDS

# Formal Language & Finite Automata

## Non-context-free languages

Dr Noleen Koehler

University of Leeds

# Reading

All reading is from the "Introduction to the theory of computation" - Micheal Sipser

- Non-context-free languages - Pages 125 - 129 (essential)

# Non-context-free languages

# Recall the pumping lemma

**Theorem (Pumping lemma for regular languages)**
*If $A$ is a regular language, then there is a number $p$ (the pumping length) where if $s$ is any string in $A$ of length at least $p$, then $s$ may be divided into three parts, $s = xyz$, satisfying the following conditions:*

1. *for each $i \geq 0$, $xy^iz \in A$,*
2. *$|y| > 0$, and*
3. *$|xy| \leq p$.*

# Pumping lemma for context-free languages

**Theorem (Pumping lemma for context-free languages)**
*If $A$ is a context-free language, then there is a number $p$ (the pumping length) where, if $s$ is any string in $A$ of length at least $p$, then $s$ may be divided into five parts, $s = uvxyz$, satisfying the following conditions:*

1. *for each $i \geq 0$, $uv^i xy^i z \in A$,*
2. *$|vy| > 0$, and*
3. *$|vxy| \leq p$.*

# Pumping lemma for context-free languages

**Proof.**
Let $A$ be a context-free language and $G = (\Sigma, \Gamma, R, S)$ a CFG that generates $A$. Let $b$ be the maximum number of symbols appearing on the right-hand side of any rule in $G$. Hence, in any parse tree of a word in $A$ every node has at most $b$ children. If a parse tree has at least $b^h$ leaves, then it must have height at least $h$ (there is one node of height 0, at most $b$ nodes of height 1, at most $b^2$ nodes of height 2, etc.). Hence, for any word of length at least $b^h$ any parse tree must have height at least $h$. We set $p = b^{|V|+1}$.
Let $s \in A$ with $|s| \geq p$ and $\tau$ be a parse tree of $s$ with a minimum number of nodes. By our observation above, we know that $\tau$ has height at least $|V| + 1$. Let $P$ be a longest root to leaf path in $\tau$ and hence $P$ has length $|V| + 1$. Hence, $P$ contains at least $|V| + 2$ nodes. All nodes on $P$ apart from the last one are labeled by a variable. By the pigeonhole principle, some variable $X$ must appear twice among the last $|V| + 1$ variables appearing on $P$.

# Pumping lemma for context-free languages

**Proof continued.**
We can now set $x$ be the word generated by the last occurrence of $X$ on $P$. Let $s'$ be the word generated by the second to last occurrence of $X$. Hence, $s'$ contains $x$ and we write $s' = vxy$. Finally, $s'$ is contained in $s$ and hence we can divide $s$ into $s = us'z$. Let $\tau_X^1$ be the subtree of $\tau$ rooted at the second to last occurrence of $X$ and $\tau_X^2$ be the subtree of $\tau$ rooted at the last occurrence of $X$. If we replace $\tau_X^1$ be $\tau_X^2$ we still obtain a valid parse tree for the word $uxz = uv^0xy^0z$. On the other hand, if we iteratively replace $\tau_X^2$ by $\tau_X^1$ exactly $i-1$ times, we obtain a valid parse tree for the word $uv^ixy^iz$. Therefore, $uv^ixy^iz \in A$ for every $i \in \mathbb{N}$.

For condition 2 observe that if $v = y = \epsilon$ then replacing $\tau_X^1$ by $\tau_X^2$ in $\tau$ yields a parse tree of $s$ with less nodes than $\tau$, a contradiction.

For condition 3 observe that $\tau_X^1$ has height at most $|V|+1$ since $X$ appears twice within the last $|V|+1$ variables of $P$. But then the word $vxy$ generated by the tree $\tau_X^1$ can have length at most $b^{|V|+1} = p$. $\square$

# $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free

**Proof.**
We proceed by contradiction. Assume that $\{a^n b^n c^n \mid n \geq 0\}$ is regular. Let $p$ be the pumping length. We choose $s = a^p b^p c^p$, clearly $s \in \{a^n b^n c^n \mid n \geq 0\}$ and as $|s| > p$ the pumping lemma says $s$ can be divided into five sections $u, v, x, y$ and $z$ such that $s = uvxyz$, where for any $i \geq 0$ the string $uv^i xy^i z \in \{a^n b^n c^n \mid n \geq 0\}$, $|vy| > 0$ and $|vxy| \leq p$. We distinguish two cases.

- Both $v$ and $y$ contain only one distinct letter. That is neither $v$ not $y$ contains $a$'s and $b$'s or $b$'s and $c$'s. In this case $uv^2xy^2z$ cannot have an equal number of $a$'s, $b$'s and $c$'s.
- Either $v$ or $y$ contains at least two distinct letters. Then in $uv^2xy^2z$ the number of $a$'s, $b$'s and $c$'s could be the same but they do not appear in the correct order (first $a$'s, then $b$'s and then $c$'s).

□